

ESPECIFICAÇÃO DETALHADA DOS REQUISITOS

RF-01 - Carregar Dados via ETL

Descrição: Como administrador do sistema, quero importar arquivos XML do Currículo Lattes para o banco relacional, para disponibilizar dados estruturados dos pesquisadores.

Atores: Sistema

Pré-condições:

1. Arquivos XML válidos (extraídos da plataforma Lattes/CNPq) devem estar depositados no diretório mapeado de entrada.
2. O banco de dados PostgreSQL deve estar ativo com as tabelas criadas.

Fluxo Principal:

1. O pipeline em Python faz a varredura do diretório de entrada e identifica um arquivo XML.
2. O sistema realiza o *parsing* do arquivo estruturado (extraindo os dados de interesse).
3. O sistema valida se o ID's já existe no banco. Se não existir, insere o registro na tabela correspondente.
4. O sistema itera sobre cada produção científica descrita no XML.
5. O sistema normaliza os dados extraídos do XML para estarem em conformidade na hora da inserção no banco.
6. O arquivo XML processado é movido para uma pasta de histórico ou marcado como concluído.

Fluxo de Exceção:

1. **Duplicidade de Pesquisador:** Caso o pesquisador já exista na base, o sistema executa uma atualização dos dados cadastrais e a data de última atualização do pesquisador, preservando o histórico existente.
2. **Duplicidade de Produções:** Para evitar duplicar artigos ou patentes de um mesmo pesquisador em cargas repetidas, o sistema valida a unicidade da produção através da combinação de (*id_pesquisador*, *título*, *ano*) ou pelo código DOI, caso esteja disponível no XML. Se houver correspondência exata, a inserção daquela linha de produção específica é ignorada.

RF-02 — Buscar Produções por Texto (Full-Text Search)

Descrição: Como usuário, quero buscar produções científicas por termos textuais, para encontrar trabalhos relacionados ao tema pesquisado.

Atores: Usuário Geral (Docentes, Alunos, Gestores).

Pré-condições: Banco de dados populado.

Fluxo Principal:

1. O usuário acessa a página inicial da interface Web e digita o termo de busca (Ex: "Inteligência Artificial") no campo correspondente.
2. O usuário opcionalmente seleciona filtros adicionais, como Ano de publicação (campo numérico ou intervalo), Tipo de Produção (Artigo, Periódico, Livro) e área de pesquisa.
3. O JavaScript realiza o disparo de uma requisição HTTP GET para o endpoint da API.
4. O Back-end em Flask traduz a string enviada para uma consulta usando funções nativas do PostgreSQL.
5. O banco avalia o *match* textual no índice GIN e calcula a relevância de palavra-chave utilizando a métrica.
6. Os registros que satisfazem o critério textual e os filtros de ano/tipo são retornados, contendo: Título, Ano, Nome do Pesquisador e Tipo da Produção.

Fluxo de Exceção:

1. **Nenhum Resultado Encontrado:** Caso o termo pesquisado ou a combinação de filtros aplicada não encontre nenhuma correspondência no índice do banco, o PostgreSQL retorna uma lista vazia. O sistema intercepta o retorno nulo disparando uma mensagem amigável na interface que orienta o usuário a refinar os termos ou remover filtros.
2. **Timeout ou Indisponibilidade do Banco de Dados:** Caso o servidor PostgreSQL esteja fora do ar, sobrecarregado ou a consulta textual estoure o limite de tempo estipulado, a camada de persistência captura a falha de conexão. O sistema interrompe a requisição, registra o erro técnico e retorna um erro de serviço indisponível (HTTP 500/503), exibindo um aviso na tela para o usuário tentar novamente mais tarde.

RF-03 — Buscar Produções Semanticamente

Descrição: Como usuário, quero realizar busca semântica nos títulos das produções, para encontrar resultados relacionados ao sentido da consulta, mesmo sem correspondência exata de termos.

Atores: Usuário Geral (Docentes, Alunos, Gestores).

Pré-condições: Modelos de IA configurados no back-end e integração com LangChain feita.

Fluxo Principal:

1. O usuário submete uma consulta em linguagem natural (Ex: "Sistemas computacionais que simulam o cérebro humano").
2. O endpoint do Flask intercepta a string de consulta e utiliza a biblioteca LangChain para enviá-la ao provedor de LLM configurado, gerando em tempo real um vetor numérico (embedding) de 1536 dimensões.
3. O back-end dispara uma consulta SQL direcionada ao PostgreSQL utilizando operadores do pgvector.
4. A similaridade vetorial é calculada com base na distância de cosseno ($\cos$) contra todos os vetores da tabela de produções.
5. Os registros mais próximos são ordenados de forma crescente de distância (ou seja, maior proximidade semântica) e retornados para compor o resultado do usuário.

Integração de Fluxo de Busca Híbrida via RRF (Casamento do RF-02 e RF-03)

Conforme definido no Projeto Arquitetural, os fluxos do RF-02 e RF-03 ocorrem de forma integrada através de um pipeline centralizado de Busca Híbrida:

1. O endpoint executa as duas buscas simultaneamente.
2. O algoritmo Reciprocal Rank Fusion (RRF) é aplicado no servidor para reordenar os resultados. O RRF pontua cada item com base na sua posição/colocação na lista do FTS e na lista Vetorial, gerando uma listagem unificada e muito mais assertiva que mitiga falhas de sinonímia e garante precisão linguística exata.

RF-04 — Consultar Perfil do Pesquisador

Descrição: Como usuário, quero visualizar o perfil de um pesquisador e suas produções, para entender suas áreas de atuação e histórico acadêmico.

Atores: Usuário Geral (Docentes, Alunos, Gestores).

Pré-condições: O pesquisador alvo deve estar previamente cadastrado via ETL.

Fluxo Principal:

1. Na tela de resultados de busca, o usuário clica no nome de um pesquisador (hiperlink mapeado com o ID interno).
2. A aplicação JS redireciona para a tela de Perfil, disparando uma requisição para o endpoint dedicado do backend.
3. A API Flask realiza duas consultas relacionais simples:
 - a. Busca dados demográficos e cadastrais do pesquisador.
 - b. Busca todas as produções científicas vinculadas à chave primária deste pesquisador.
4. O back-end monta um payload JSON estruturado contendo: Informações pessoais do pesquisador e a lista completa de produções associadas.
5. O Front-end renderiza os componentes na tela de forma dinâmica.

Fluxo de Exceção:

1. **Pesquisador Não Encontrado:** Caso o usuário tente acessar o perfil de um pesquisador informando um identificador que não exista na base de dados (seja por digitação direta na URL ou por exclusão recente do registro), a consulta ao banco de dados retornará vazia. A API aborta o processamento, retorna um erro de recurso não encontrado (HTTP 404) e a interface exibe uma tela de alerta informando que o perfil do pesquisador solicitado não foi localizado no sistema.
2. **Pesquisador Sem Produções Científicas Cadastradas:** Caso o pesquisador exista na base de dados, mas não possua nenhuma linha de produção vinculada a ele (por exemplo, um perfil recém-importado cujo XML original não continha artigos ou patentes), a consulta de dados demográficos retorna com sucesso, mas a lista de produções vem vazia. O sistema exibe uma mensagem informativa na seção de histórico (Ex: *"Este pesquisador não possui produções científicas registradas até o momento."*).
3. **Filtros Inexistentes ou Sem Correspondência no Perfil:** Caso o usuário aplique filtros por Ano ou Tipo dentro da tela de perfil do pesquisador e nenhum registro atenda aos critérios escolhidos, a API (ou a filtragem via JavaScript no front-end) retorna um array vazio. O perfil e os dados cadastrais do docente continuam visíveis na tela, mas a listagem de produções é ocultada temporariamente para dar lugar a um aviso indicando que não há trabalhos para o filtro selecionado, permitindo que o usuário limpe os filtros para restaurar a lista completa.

RF-05 — Gerar Dados Analíticos e CSV

Descrição: Como usuário, quero acessar uma área de modo analítico integrada na interface web para visualizar gráficos consolidados de produção científica e exportar os dados brutos processados em formato CSV.

Atores: Gestor Acadêmico / Administrador.

Pré-condições: Registros de produção científica e currículos Lattes devidamente extraídos e armazenados no banco PostgreSQL.

Fluxo Principal:

1. O usuário navega até a seção ou aba de **"Painel Analítico"**.
2. O front-end dispara uma requisição de leitura para os endpoints analíticos do back-end Flask durante o carregamento da tela.
3. O back-end Flask aciona a camada de engenharia de dados, utilizando as bibliotecas pandas e numpy para estruturar e consolidar as visões analíticas agregadas:
 - a. **Agrupamento por Pesquisador:** Consolidação de ID Lattes, nome e grande área de atuação.
 - b. **Agrupamento Temporal:** Mapeamento dinâmico dos anos de produção e cálculo automático do **quadriênio correspondente da CAPES**.
 - c. **Contagem de Produção:** Cruzamento das chaves gerando matrizes de fato (Pesquisador, Período, Tipo de Produção e Volume Científico).
4. O módulo de análise utiliza o matplotlib (ou seaborn) para computar e plotar os gráficos de alta fidelidade baseados nos DataFrames estruturados.
5. O Flask converte a imagem do gráfico gerado em memória para uma string codificada em Base64 e envia no payload JSON de resposta (ou gera uma rota de imagem estática), permitindo que o front-end a renderize nativamente em tags.
6. A interface exibe o painel completo de gráficos de forma nativa e integrada.
7. Na mesma tela, caso o usuário queira os dados tabulares, ele clica no botão **"Exportar CSV Analítico"**.
8. O servidor gera os arquivos formatados no padrão CSV (delimitados por ponto e vírgula, com codificação UTF-8 com BOM para preservar acentuações e compatibilidade imediata com o Microsoft Excel).
9. O download é disponibilizado imediatamente no navegador.

Restrições de Formato:

1. **Alinhamento Visual (UI/UX):** Os gráficos gerados pelo Matplotlib devem adotar paletas de cores e estilos pré-definidos no figma..
2. **Integridade do CSV:** Títulos de trabalhos científicos ou nomes de pesquisadores que contenham ponto e vírgula devem ser obrigatoriamente tratados, evitando a quebra de colunas no arquivo final.